

# Serving = four concerns, not a stack

*orthogonal decisions - they interact, but none sits on top of another*

**Request stream**  
(text → tokens)

## THE FOUR SERVING CONCERNS

### REUSE

prefix cache: reuse  
KV of shared context

*re-prefill FLOPs skipped*

### REQUEST SCHEDULING

continuous batching:  
admit/evict each step

*latency ↔ utilization*

### RESOURCE SPECIALISATION

P/D split: prefill pool  
+ decode pool

*two regimes → two pools*

*all four in parallel*

### STATE MANAGEMENT

KV cache in fixed pages  
(PagedAttention)

*memory capacity → concurrency*

**Output tokens**